

Tecnicatura Universitaria en Programación a Distancia

Metodología de Sistemas II

Comisión 15

Trabajo Práctico Integrador (TPI)

Agenda de turnos del Hospital Central:
diagnóstico, refactoring y revisión de código

Alumnos:

Agustín Emiliano Sotelo Carmelich (agustinemiliano22@gmail.com)

Bruno Giuliano Vapore (brunogvapore@gmail.com)

Santiago Octavio Varela (santiago.varela@tupad.utn.edu.ar)

8 de Noviembre de 2026

Repositorio: https://github.com/enkai12/hospital-central-msii

Link al video: https://www.youtube.com/watch?v=JWyaUtaGNEY

Resumen ejecutivo

- *Resumen ejecutivo de 5 a 7 líneas.*

Analizamos una agenda de turnos en Python con persistencia en JSON que funciona bien para el usuario, pero presenta funciones largas, código repetido y responsabilidades mezcladas. Mediante lectura del código, casos de borde y SonarQube diagnosticamos once hallazgos de deuda técnica con su impacto y prioridad. Sobre esa base aplicamos refactorings seguros en Pull Requests independientes, validados con revisión cruzada, comparación de la salida y, desde que la incorporamos, integración continua. Al cierre, pasan las 20 pruebas automatizadas, Black y flake8, se conserva idéntica la salida de la demo y SonarQube registra cero problemas.

1. Descripción del sistema base

> Etapa 1. Selección y descripción del sistema base

Describir el dominio, alcance, actores, funcionalidades, tecnología utilizada y versión inicial del sistema. Incluir enlace al repositorio y evidencia de funcionamiento.

Elegimos una agenda de turnos para la recepción del Hospital Central. Armamos un prototipo nuevo en Python y lo tomamos como versión inicial.

Quien usa el sistema es el personal de recepción: registra pacientes, reserva y cancela turnos y consulta las agendas. Los profesionales no operan el sistema; vienen precargados con su especialidad. Trabajamos con personas ficticias y números internos sin DNI, historias clínicas ni datos médicos.

Entidad	Datos	Para qué sirve
Paciente	Identificador interno y nombre	Identificar a quien reserva el turno.
Profesional	Identificador, nombre y especialidad	Identificar la agenda que se consulta o reserva.
Turno	Identificador, paciente, profesional, fecha, horario y estado	Registrar una reserva y conservar su historial.

Operación	Resultado esperado
Registrar paciente	Se incorpora un paciente con identificador único.
Reservar turno	Se crea una reserva activa si los datos y el horario son válidos.
Cancelar turno	El turno pasa a cancelado y se libera la franja.
Consultar agenda	Se muestran los turnos de un profesional en una fecha y las franjas libres.
Consultar turnos de un paciente	Se muestra su historial, distinguiendo activos y cancelados.

Cada turno ocupa una franja de 30 minutos; hay seis por día, que empiezan entre las 09:00 y las 11:30. No puede haber dos turnos activos del mismo profesional en la misma fecha y franja, ni dos del mismo paciente aunque sean con profesionales distintos. Cancelar deja el turno como CANCELADO y libera la franja, y una operación rechazada no cambia nada de lo guardado. Estas reglas son el comportamiento que comparamos antes y después de los refactorings.

El sistema funciona con un menú de consola, guarda los datos en un archivo JSON y usa turnos de duración fija. Facturación, obras sociales, urgencias, notificaciones, autenticación e interfaz web quedan fuera del alcance.

Lo desarrollamos en Python 3.13, solo con la biblioteca estándar, y usamos Black y flake8 para el formato y el estilo.

Repositorio:

- **https://github.com/enkai12/hospital-central-msii**

Versión inicial: tag estado-inicial, commit e5d6927

- **https://github.com/enkai12/hospital-central-msii/tree/estado-inicial**

Evidencia:

1. Opción 5, profesional 1, fecha 2026-11-10: las seis franjas están libres.

```
PS C:\Users\Bruno\Desktop\14 - Metodología de Sistemas II\TPI> python main.py

Hospital Central - Agenda de turnos
1. Registrar paciente
2. Listar pacientes
3. Reservar turno
4. Cancelar turno
5. Consultar la agenda de un profesional
6. Consultar los turnos de un paciente
0. Salir
Opción: 5
Número de profesional: 1
Fecha (AAAA-MM-DD): 2026-11-10
09:00  libre
09:30  libre
10:00  libre
10:30  libre
11:00  libre
11:30  libre
```

2. Opción 3: paciente 1 (Ana Gómez), profesional 1, 2026-11-10, 09:00. Se reserva el turno 1.

```
Hospital Central - Agenda de turnos
1. Registrar paciente
2. Listar pacientes
3. Reservar turno
4. Cancelar turno
5. Consultar la agenda de un profesional
6. Consultar los turnos de un paciente
0. Salir
Opción: 3
1. Laura Méndez (Clínica médica)
2. Martín Ríos (Pediatría)
3. Sofía Herrera (Traumatología)
4. Diego Paz (Cardiología)
Número de paciente: 1
Número de profesional: 1
Fecha (AAAA-MM-DD): 2026-11-10
Horario (HH:MM): 09:00
Turno 1 reservado: Ana Gómez con Laura Méndez el 2026-11-10 a las 09:00.
```

3. Opción 3: paciente 2 (Carlos Ruiz), profesional 1, misma fecha y horario. Se rechaza porque el profesional está ocupado.

```
Hospital Central - Agenda de turnos
1. Registrar paciente
2. Listar pacientes
3. Reservar turno
4. Cancelar turno
5. Consultar la agenda de un profesional
6. Consultar los turnos de un paciente
0. Salir
Opción: 3
1. Laura Méndez (Clínica médica)
2. Martín Ríos (Pediatría)
3. Sofía Herrera (Traumatología)
4. Diego Paz (Cardiología)
Número de paciente: 2
Número de profesional: 1
Fecha (AAAA-MM-DD): 2026-11-10
Horario (HH:MM): 09:00
El profesional ya tiene un turno en esa fecha y horario.
```

4. Opción 3: paciente 1, profesional 2, misma fecha y horario. Se rechaza porque Ana ya tiene un turno a esa hora.

```
Hospital Central - Agenda de turnos
1. Registrar paciente
2. Listar pacientes
3. Reservar turno
4. Cancelar turno
5. Consultar la agenda de un profesional
6. Consultar los turnos de un paciente
0. Salir
Opción: 3
1. Laura Méndez (Clínica médica)
2. Martín Ríos (Pediatría)
3. Sofía Herrera (Traumatología)
4. Diego Paz (Cardiología)
Número de paciente: 1
Número de profesional: 2
Fecha (AAAA-MM-DD): 2026-11-10
Horario (HH:MM): 09:00
El paciente ya tiene un turno en esa fecha y horario.
```

5. Opción 4: turno 1. Queda cancelado.

```
Hospital Central - Agenda de turnos
1. Registrar paciente
2. Listar pacientes
3. Reservar turno
4. Cancelar turno
5. Consultar la agenda de un profesional
6. Consultar los turnos de un paciente
0. Salir
Opción: 4
Número de turno: 1
Turno 1 cancelado. La franja quedó libre.
```

6. Opción 3: paciente 2, profesional 1, 2026-11-10, 09:00. Ahora se reserva (turno 2).

```
Hospital Central - Agenda de turnos
1. Registrar paciente
2. Listar pacientes
3. Reservar turno
4. Cancelar turno
5. Consultar la agenda de un profesional
6. Consultar los turnos de un paciente
0. Salir
Opción: 3
1. Laura Méndez (Clínica médica)
2. Martín Ríos (Pediatría)
3. Sofía Herrera (Traumatología)
4. Diego Paz (Cardiología)
Número de paciente: 2
Número de profesional: 1
Fecha (AAAA-MM-DD): 2026-11-10
Horario (HH:MM): 09:00
Turno 2 reservado: Carlos Ruiz con Laura Méndez el 2026-11-10 a las 09:00.
```

7. Opción 0 para salir, volver a ejecutar python main.py y consultar la opción 5 y la opción 6 con el paciente 1: el turno 2 sigue en la agenda y el turno 1 aparece cancelado en el historial de Ana.

```
Hospital Central - Agenda de turnos
1. Registrar paciente
2. Listar pacientes
3. Reservar turno
4. Cancelar turno
5. Consultar la agenda de un profesional
6. Consultar los turnos de un paciente
0. Salir
Opción: 0
Hasta luego.
PS C:\Users\Bruno\Desktop\14 - Metodología de Sistemas II\TPI>
```

```
Hospital Central - Agenda de turnos
1. Registrar paciente
2. Listar pacientes
3. Reservar turno
4. Cancelar turno
5. Consultar la agenda de un profesional
6. Consultar los turnos de un paciente
0. Salir
Opción: 5
Número de profesional: 1
Fecha (AAAA-MM-DD): 2026-11-10
09:00  turno 2 - Carlos Ruiz
09:30  libre
10:00  libre
10:30  libre
11:00  libre
11:30  libre
```

```
Hospital Central - Agenda de turnos
1. Registrar paciente
2. Listar pacientes
3. Reservar turno
4. Cancelar turno
5. Consultar la agenda de un profesional
6. Consultar los turnos de un paciente
0. Salir
Opción: 6
Número de paciente: 1
Turno 1: 2026-11-10 09:00 con Laura Méndez - CANCELADO
```

2. Calidad externa e interna

> Etapa 2. Calidad externa vs. calidad interna

> *Explicar que funciona correctamente desde la mirada del usuario y que problemas existen en la estructura interna. Relacionar funcionalidad visible, mantenibilidad, legibilidad y evolución futura.*

Desde la mirada de quien usa el sistema, la versión inicial funciona bien. El personal de recepción puede registrar pacientes, reservar y cancelar turnos y consultar la agenda de un profesional o el historial de un paciente. Las reglas acordadas se cumplen: se rechazan las fechas inválidas, los horarios fuera de las franjas y los turnos que se superponen, tanto para el profesional como para el paciente. Cancelar libera la franja, las consultas salen ordenadas y los datos se conservan al cerrar y volver a abrir el programa. Recorrimos la demo completa y todos los resultados coincidieron con lo esperado.

La estructura interna, en cambio, tiene problemas que no se ven desde la consola. La función principal del menú tiene 91 líneas: pide los datos, los valida, muestra los resultados y decide cuándo guardar. La búsqueda de un paciente, un profesional o un turno por su número está escrita siete veces, y la validación de la fecha, dos. El mismo módulo contiene las reglas de los turnos y la lectura y escritura del archivo JSON. Además, las funciones informan los errores devolviendo False y un mensaje, en lugar de lanzar una excepción, por lo que cada opción del menú tiene que revisar el resultado antes de seguir. Tampoco hay pruebas automatizadas: todo se comprueba a mano desde la consola.

Aspecto	Qué vemos	Cómo afecta
Funcionalidad visible	Las cinco operaciones y las reglas se cumplen.	El sistema es útil tal como está.
Legibilidad	La función del menú es larga y los bucles de búsqueda usan nombres como p y t.	Cuesta seguir el flujo y ubicar dónde se decide cada cosa.
Mantenibilidad	La misma búsqueda y la misma validación están copiadas en varias funciones.	Una corrección hay que hacerla en todos los lugares, y es fácil olvidarse de uno.

Aspecto	Qué vemos	Cómo afecta
Evolución futura	Las reglas y el manejo del archivo JSON están en el mismo módulo, y cada opción nueva se agrega dentro de la misma función.	Cambiar la forma de guardar o sumar una operación obliga a tocar código que ya funciona.

En el uso normal el programa funciona bien; los fallos aparecen en casos de borde (H4, H9 y H11). El problema principal es que cada cambio cuesta más y tiene más riesgo. Por eso el trabajo no apunta a agregar funciones, sino a mejorar la estructura interna conservando exactamente el mismo comportamiento visible.

3. Diagnóstico de deuda técnica y code smells

Etapa 3. Deuda técnica y code smells

Revisamos el código de la versión inicial con tres fuentes: la lectura del código, la ejecución de casos de borde y el análisis estático. Encontramos once problemas. Los ordenamos por prioridad, y algunos se resuelven con el mismo cambio: por ejemplo, la validación de fecha duplicada (H5) se corrige al extraer los controles de la reserva (H1).

ID	Problema	Ubicación	Prioridad
H1	Funciones largas	`main.py: main()` (líneas 27–117); `gestion.py: reservar_turno` (líneas 39–86)	**Alta**
H2	Búsqueda por número duplicada	`gestion.py`, líneas 41, 48, 91, 105, 128, 137 y 146	**Alta**
H3	Reglas y archivo en el mismo módulo	`gestion.py: cargar_datos` y `guardar_datos` (12–21) junto a las reglas (24–150)	**Alta**
H4	Errores sin controlar al cargar o leer	`main.py`, líneas 28–33 y 37; `gestion.py`, líneas 12–16	**Alta**
H5	Validación de fecha duplicada	`gestion.py`, líneas 54–59 y 111–116	**Media**
H6	Demasiados parámetros en la reserva	`gestion.py`, línea 39; `main.py`, líneas 57–66	**Media**
H7	Errores informados con False	`gestion.py`: 14 return False; `main.py`: 5 controles de ok	**Media**
H8	La consola depende de la forma interna de los datos	`main.py`, líneas 52–56, 89–96 y 108–112	**Media**
H9	Escritura directa sobre el archivo	`gestion.py`, líneas 19–21	**Media**
H10	Datos del dominio como texto suelto	`gestion.py`, líneas 8–9 y 73–80	**Baja**

ID	Problema	Ubicación	Prioridad
H11	Agenda y reserva no coinciden con datos inconsistentes	`gestion.py`, líneas 121–131	**Baja**

ID	Code smell / problema	Detalles (evidencia, impacto y riesgo)	Prioridad y acción propuesta
H1	**Funciones largas (Long Method)**	**Evidencia:** `main()` tiene 91 líneas y una rama if/elif por cada opción del menú. En cada rama pide los datos, los convierte, llama a la regla, decide si guardar y muestra el resultado. `reservar_turno` tiene 48 líneas y hace seis controles seguidos. **Impacto:** para entender una sola opción hay que leer toda la función, y los controles de la reserva no se pueden probar por separado. **Riesgo de mantenimiento:** alto; cada operación nueva se agrega dentro de la misma función y la hace más larga.	**Prioridad: Alta** **Acción propuesta:** extraer una función por opción del menú y una por control de la reserva.
H2	**Búsqueda por número duplicada (Duplicated Code)**	**Evidencia:** el mismo bucle que recorre una lista hasta encontrar un número aparece siete veces: tres para pacientes, tres para profesionales y una para turnos. Además, el bucle no se corta al encontrar el elemento. **Impacto:** cualquier cambio en la forma de buscar, por ejemplo cortar el bucle o avisar si el número no existe, hay que repetirlo en siete lugares. **Riesgo de mantenimiento: alto;** es fácil corregir uno y olvidarse de los demás.	**Prioridad: Alta** **Acción propuesta:** una sola función de búsqueda, reutilizada por todas las operaciones.
H3	**Reglas y archivo en el mismo módulo (baja cohesión: dos motivos para cambiar)**	**Evidencia:** `gestion.py` tiene las reglas de los turnos y también la lectura y escritura del JSON, con `json` y `shutil`. **Impacto:** el módulo cambia por dos motivos distintos, si cambia una regla o si cambia la forma de guardar, y para	**Prioridad: Alta** **Acción propuesta:** mover la carga y el guardado a un módulo de

ID	Code smell / problema	Detalles (evidencia, impacto y riesgo)	Prioridad y acción propuesta
		probar una regla hay que conocer el archivo. **Riesgo de mantenimiento: alto;** cambiar el almacenamiento obliga a tocar el código de las reglas.	persistencia propio.
H4	**Errores sin controlar al cargar o leer**	**Evidencia:** solo se controla el JSON mal escrito. Si el archivo es JSON válido pero con otra forma, el programa se corta en la primera operación: con TypeError si el archivo tiene [] y con KeyError si tiene {}. Si faltan los dos archivos, se corta con `FileNotFoundError`. Si se cierra la entrada con Ctrl+D, termina con `EOFError`. **Impacto:** afecta la calidad externa, porque quien usa el sistema ve un error técnico en lugar de un aviso. **Riesgo de mantenimiento: medio;** los fallos aparecen lejos de su causa y cuesta encontrarlos.	**Prioridad: Alta** **Acción propuesta:** validar la estructura al cargar y atrapar los errores previstos con un mensaje claro.
H5	**Validación de fecha duplicada (Duplicated Code)**	**Evidencia:** `reservar_turno` y `agenda_profesional` repiten las mismas seis líneas para comprobar el formato AAAA-MM-DD. **Impacto:** si cambia el formato aceptado, hay que corregir las dos copias. **Riesgo de mantenimiento:** medio; hoy son dos copias, pero cada consulta nueva con fecha agregaría otra.	**Prioridad: Media** **Acción propuesta:** una función `validar_fecha`.
H6	**Demasiados parámetros en la reserva (Long Parameter List)**	**Evidencia:** `reservar_turno` recibe cinco parámetros, y cuatro de ellos (paciente, profesional, fecha y hora) viajan siempre juntos desde la consola. **Impacto:** la firma es larga y es fácil pasar los datos en otro orden sin que nada lo detecte.	**Prioridad: Media** **Acción propuesta:** agrupar esos cuatro datos en un solo objeto de solicitud de turno.

ID	Code smell / problema	Detalles (evidencia, impacto y riesgo)	Prioridad y acción propuesta
		Riesgo de mantenimiento: medio; sumar un dato a la reserva obliga a cambiar la firma y todas sus llamadas.	
H7	**Errores informados con False**	**Evidencia:** las operaciones devuelven `(False, mensaje)` cuando algo falla; hay 14 retornos así. En las consultas, el segundo valor es un mensaje si falló y una lista si salió bien. **Impacto:** quien llama tiene que revisar el resultado cada vez antes de seguir, y el mismo valor puede ser de dos tipos distintos. **Riesgo de mantenimiento: medio;** si una nueva llamada se olvida de revisar el resultado, el error pasa sin aviso.	**Prioridad:** Media **Acción propuesta:** lanzar excepciones propias del dominio y atraparlas en la consola.
H8	**La consola depende de la forma interna de los datos (acoplamiento)**	**Evidencia:** `main.py` recorre `datos["profesionales"]` directamente y arma los mensajes leyendo claves internas como `franja["turno"]["turno_id"]`. **Impacto:** cualquier cambio en la estructura de los datos rompe la consola, aunque las reglas sigan iguales. **Riesgo de mantenimiento:** medio; obliga a cambiar dos capas por un solo motivo.	**Prioridad:** Media **Acción propuesta:** que la consola use solo funciones de consulta y no lea el diccionario directamente.
H9	**Escritura directa sobre el archivo**	**Evidencia:** `guardar_datos` abre `datos/local.json` en modo escritura y lo reemplaza directamente. **Impacto:** si la escritura se interrumpe a mitad, el archivo queda incompleto y en la próxima apertura se informa como dañado. **Riesgo de mantenimiento:** medio; es un riesgo de pérdida de datos, poco probable pero grave.	**Prioridad:** Media **Acción propuesta:** escribir en un archivo temporal y reemplazar el original al terminar.
H10	**Datos del dominio como texto suelto**	**Evidencia:** el turno es un diccionario, su estado es el texto "ACTIVO" o	**Prioridad:** Baja

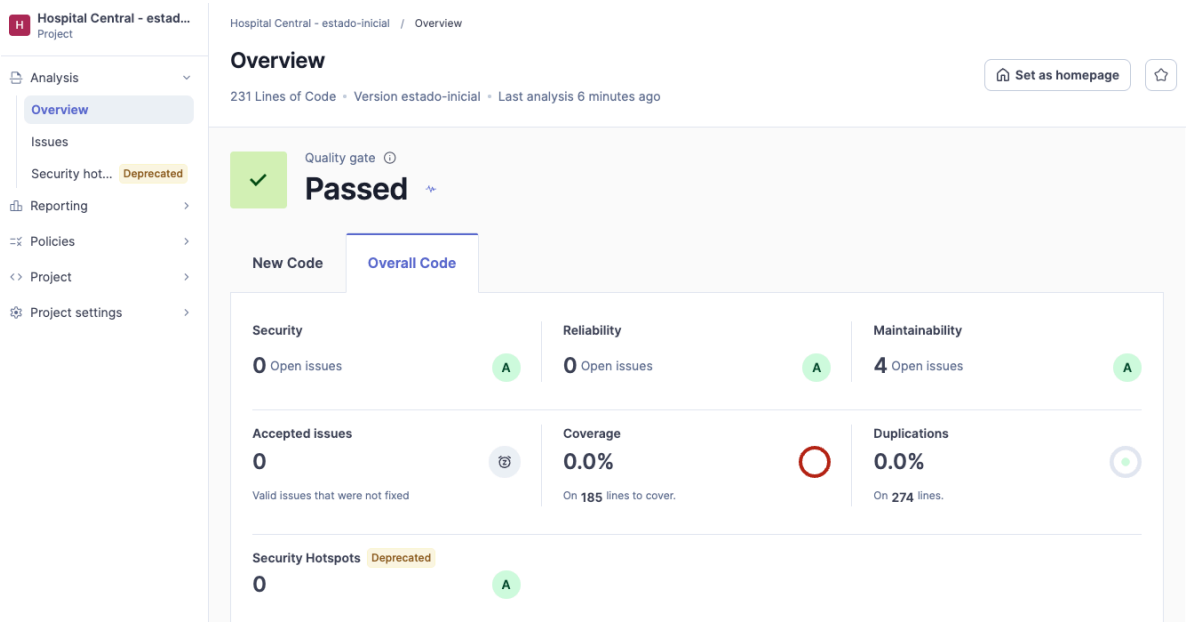
ID	Code smell / problema	Detalles (evidencia, impacto y riesgo)	Prioridad y acción propuesta
	(Primitive Obsession)	"CANCELADO" y la fecha es un texto que se vuelve a validar en cada función que la recibe. **Impacto:** nada impide que en algún lugar se escriba otro estado o una fecha sin validar. **Riesgo de mantenimiento:** bajo hoy, porque los estados ya son constantes, pero crece si se agregan estados.	**Acción propuesta:** representar el turno y su estado con tipos propios, si el resto de los cambios lo justifica.
H11	**Agenda y reserva no coinciden con datos inconsistentes**	**Evidencia:** si un turno activo apunta a un paciente que no existe (por ejemplo, porque se editó el archivo a mano), la agenda muestra esa franja como libre, pero al intentar reservar se rechaza porque el profesional está ocupado. **Impacto:** el sistema da dos respuestas distintas sobre la misma franja. **Riesgo de mantenimiento:** bajo; solo pasa con un archivo modificado por fuera del programa.	**Prioridad: Baja** **Acción propuesta:** que la agenda marque la franja como ocupada aunque falte el paciente, y validar las referencias al cargar.

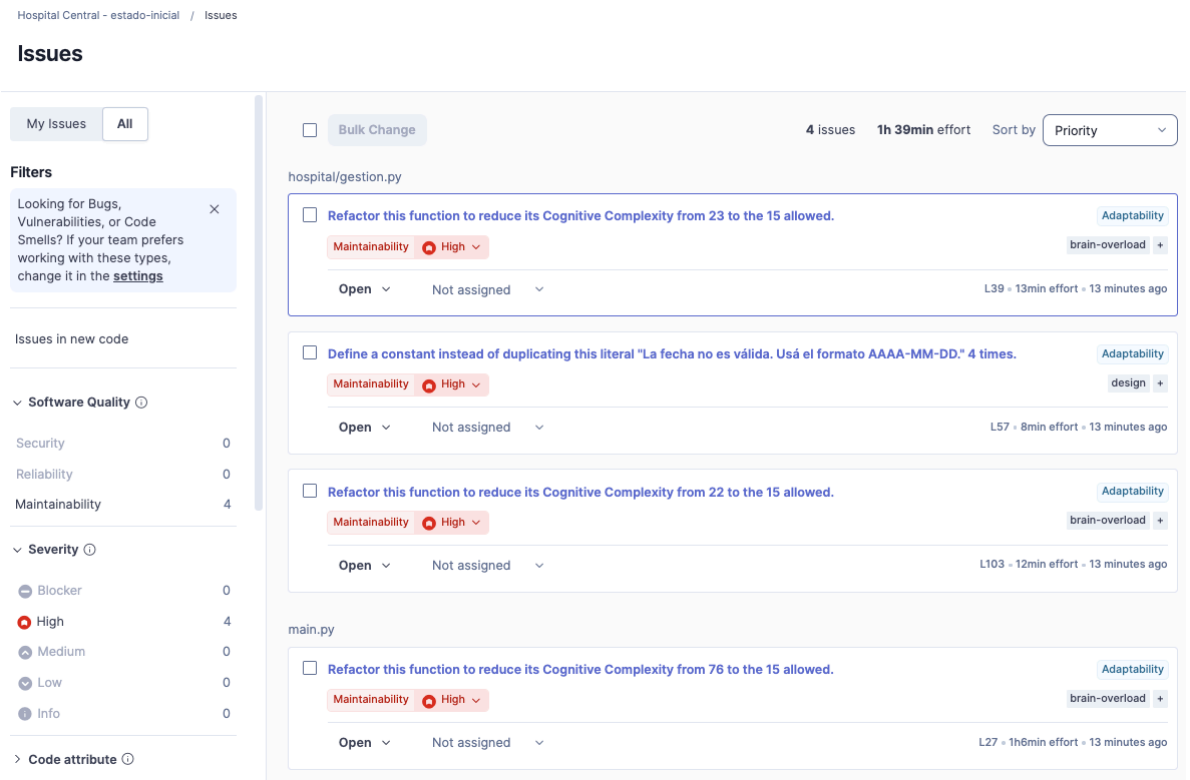
Para completar la revisión pasamos SonarQube sobre la versión inicial (tag estado-inicial). No encontró errores ni vulnerabilidades y el proyecto pasó el quality gate por defecto (Sonar way), pero marcó cuatro problemas de mantenibilidad que coinciden con nuestro diagnóstico:

- main() tiene una complejidad cognitiva de 76, cuando el máximo que acepta la herramienta es 15. Sube tanto porque cada opción del menú es una rama if/elif con sus propios controles adentro (H1).
- reservar_turno() tiene 23, por los seis controles que hace uno detrás de otro (H1).
- agenda_profesional() tiene 22: combina búsquedas anidadas por número con la validación de la fecha (H1, H2 y H5).
- El mensaje de error de la fecha está escrito cuatro veces en gestion.py, porque la validación no está centralizada (H5).

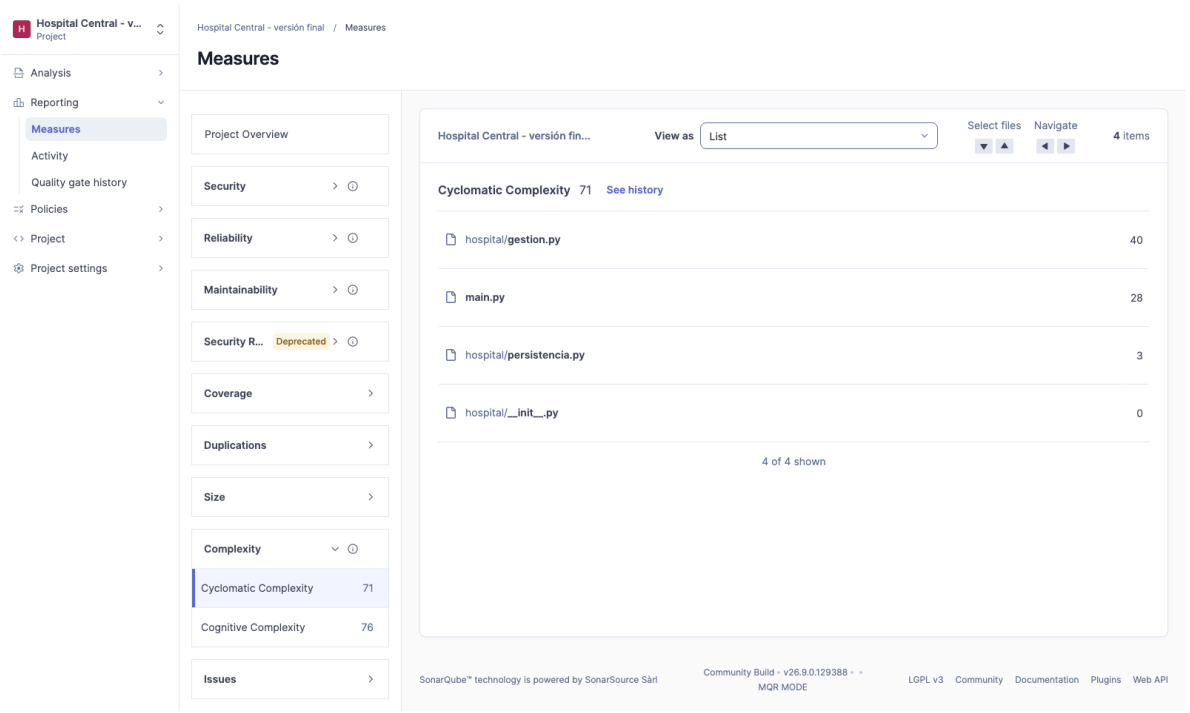
Aun así, el proyecto obtuvo la calificación A en mantenibilidad. SonarQube calcula esa nota comparando el tiempo estimado para corregir los problemas (99 minutos) con el que llevaría escribir el código de nuevo, y en un programa de 231 líneas esa proporción es

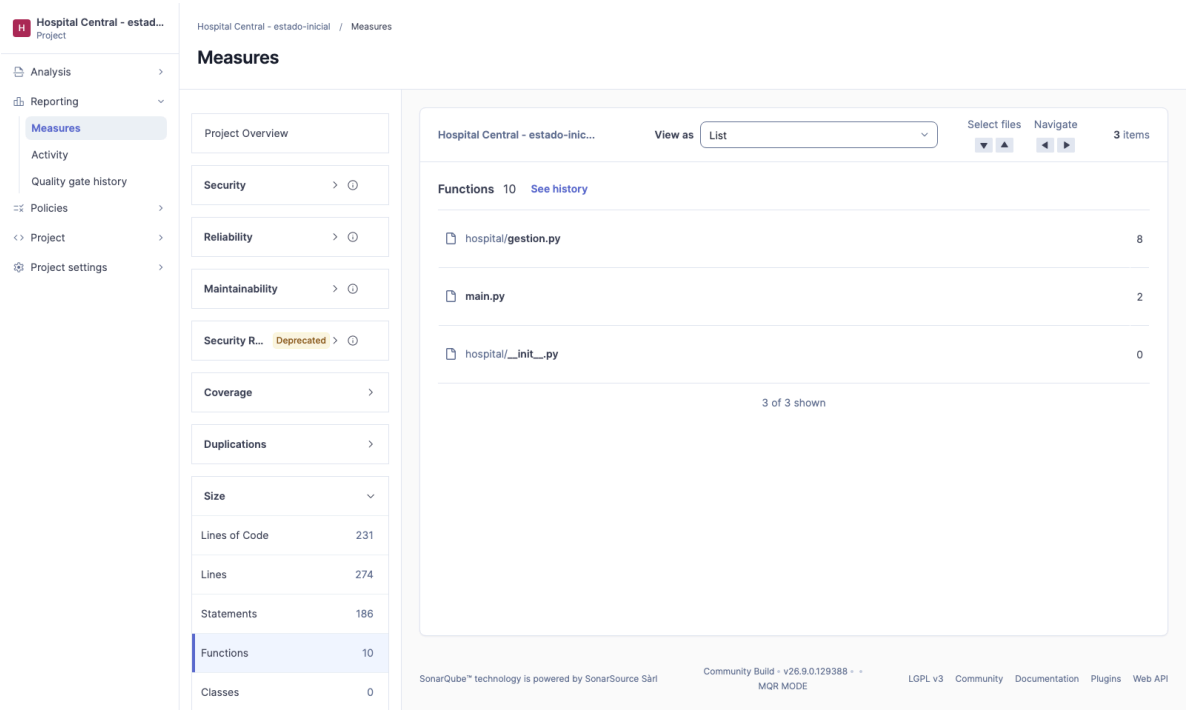
baja. Además, sus reglas analizan cada función por separado: no detectan que gestion.py mezcle las reglas con el archivo (H3) ni que la consola dependa de la forma interna de los datos (H8). Tampoco tomó como duplicados los bucles de búsqueda de H2, porque son cortos y cada copia recorre una lista distinta. Por eso usamos el análisis estático como complemento de la lectura del código y de las pruebas de casos de borde, no como única fuente.





Arriba, la pantalla principal del análisis de SonarQube sobre estado-inicial; abajo, los cuatro problemas de mantenibilidad. Las capturas y el diagnóstico completo también están en docs/ del repositorio.





Las capturas de arriba muestran el tamaño y la complejidad de la versión inicial: 231 líneas de código y una complejidad ciclomática total de 69.

4. Aplicación de SOLID y Clean Code

> Etapa 4. Aplicación de SOLID y Clean Code
>
> *Proponer y aplicar mejoras vinculadas a SRP, OCP, LSP, ISP y DIP cuando corresponda. Mejorar nombres, funciones, manejo de errores, cohesión, acoplamiento y límites entre componentes.*

Aplicamos los principios que corresponden a un programa chico, escrito principalmente con funciones y una única clase de datos inmutable (SolicitudTurno) para agrupar los parámetros de la reserva. Los que no aplican los explicamos igual, porque forzarlos agregaría complejidad sin beneficio.

Principio	Cómo lo aplicamos
SRP (responsabilidad única)	gestion.py tenía dos motivos para cambiar: las reglas de los turnos y la forma de guardar. Llevamos la carga y el guardado a un módulo de persistencia propio (H3, PR #21), y en main() cada opción del menú pasa a tener su propia función (H1, PR #23).
OCP (abierto/cerrado)	Con una función por opción y el diccionario OPCIONES en main.py, sumar una operación nueva es agregar una función y su entrada en el menú, sin tocar las demás opciones. No llega a cumplir OCP del todo, porque el menú se sigue modificando, pero el cambio queda acotado a una función nueva y su entrada en OPCIONES (H1, PR #23).
LSP (sustitución de Liskov)	No aplica: no hay herencia ni subtipos que tengan que poder reemplazarse entre sí.
ISP (segregación de interfaces)	No aplica: no hay interfaces ni clases que obliguen a implementar métodos que no se usan.
DIP (inversión de dependencias)	No lo aplicamos de forma estricta. Después de H3, las reglas no conocen el archivo y el único módulo que lo toca es el de persistencia. Agregar una interfaz entre los dos sería sobrediseño para un programa de este tamaño.

En cuanto a Clean Code, trabajamos sobre estos puntos:

Tema	Antes	Después
Nombres	Los bucles de búsqueda usaban p y t.	Se llaman paciente, profesional y turno (H2).
Duplicación	La misma búsqueda por número aparecía siete veces.	Una sola función, buscar_por_id (H2).
Funciones	main() tenía 91 líneas y reservar_turno hacía seis controles seguidos.	Una función por cada opción del menú en `main.py` y una función por cada control de la reserva y de la fecha en `gestion.py` (H1 y H5).
Cohesión y límites	Las reglas y el archivo JSON estaban en el mismo módulo.	gestion.py tiene solo las reglas, y persistencia.py es el único módulo que toca el archivo (H3).
Parámetros	reservar_turno recibía cinco parámetros.	Se agrupan en el objeto inmutable SolicitudTurno (@dataclass(frozen=True)), por lo que reservar_turno y buscar_conflicto reciben una sola estructura en lugar de cuatro parámetros sueltos (H6, PR #26).
Manejo de errores	Las funciones devuelven (False, mensaje) en lugar de lanzar excepciones (H7).	En persistencia.py, el guardado escribe primero en un archivo temporal (local.json.tmp) y reemplaza al original de forma atómica con os.replace, evitando corromper el archivo ante fallos (H9, PR #25). El reemplazo de (False, mensaje) por excepciones propias (H7) queda como mejora futura para no alterar la interfaz entre capas.
Acoplamiento	La consola lee directamente el diccionario de datos (H8).	Queda como mejora futura: aislar la lectura directa del diccionario en la consola mediante funciones o vistas específicas del dominio (H8).

5. Refactorings aplicados con evidencia antes/después

> Etapa 5. Refactoring seguro
>
> *Aplicar al menos 4 refactorings documentados. Deben incluir, como mínimo, Extract Method, Move Method o reubicación de responsabilidad, e Introduce Parameter Object o simplificación equivalente de parámetros.*

Hicimos cada refactoring en una rama y un Pull Request propios, y lo comprobamos antes y después de cambiarlo. Ninguno cambia el comportamiento visible: si al refactorizar aparecía un error, lo dejamos igual y lo tratamos aparte.

Hallazgo	Refactoring	Issue y Pull Request	Estado
H2. Búsqueda por número duplicada	Extract Method sobre el código duplicado	Issue #15, PR #20	Integrado en main
H1. Funciones largas	Extract Method	Issue #17, PR #23	Integrado en main
H3. Reglas y archivo en el mismo módulo	Move Method (reubicación de responsabilidad)	Issue #18, PR #21	Integrado en main
H6. Demasiados parámetros en la reserva	Introduce Parameter Object	Issue #19, PR #26	Integrado en main

Refactoring 1: búsqueda única (H2)

El mismo bucle de búsqueda por número estaba copiado siete veces en gestion.py, y además seguía recorriendo la lista después de encontrar el elemento. Antes, cada operación buscaba así:

```python
paciente = None
for p in datos["pacientes"]:
   if p["id"] == paciente_id:
       paciente = p
```

Ahora hay una sola función, que corta en cuanto encuentra el elemento, y las siete búsquedas la usan:

```python
def buscar_por_id(elementos, id_buscado):
    for elemento in elementos:
        if elemento["id"] == id_buscado:
            return elemento
    return None

paciente = buscar_por_id(datos["pacientes"], paciente_id)
```

Conservamos el comportamiento con datos inconsistentes: si un turno apunta a un paciente que no existe, la agenda sigue mostrando la franja libre (H11), porque corregirlo cambiaría lo que ve el usuario. gestion.py pasó de 150 a 145 líneas, y las búsquedas ya no usan nombres como p y t.

Verificación: corrimos la demo completa y dieciséis casos de borde antes y después del cambio, y la salida de la consola y el archivo final fueron idénticos. Pasan Black y flake8.

Commit e3f1d16, PR #20, revisado e integrado por Bruno. Las pruebas automatizadas todavía no existían; las 20 pasan sobre la versión final.

Refactoring 2: funciones más cortas y validación de fecha única (H1 y H5)

En la versión inicial, main() tenía 91 líneas con una estructura if/elif que concentraba la lectura de datos, la llamada a las reglas, el guardado y la impresión de resultados (complejidad cognitiva de 76 en SonarQube). Por su parte, reservar_turno() tenía 48 líneas con seis validaciones seguidas (complejidad de 23) y repetía las mismas líneas que agenda_profesional() para validar el formato de la fecha (H5). Antes, reservar_turno validaba la fecha y los conflictos de horario en línea dentro de la misma función:

```python
try:
    fecha_valida = datetime.strptime(fecha, "%Y-%m-%d")
except ValueError:
    return False, "La fecha no es válida. Usá el formato AAAA-MM-DD."
if fecha_valida.strftime("%Y-%m-%d") != fecha:
    return False, "La fecha no es válida. Usá el formato AAAA-MM-DD."

if hora not in FRANJAS:
```

```
    return False, "El horario tiene que ser una de las franjas: " + ", ".join(
        FRANJAS
    )

for turno in datos["turnos"]:
    if turno["estado"] == ACTIVO and turno["fecha"] == fecha:
        if turno["hora"] == hora and turno["profesional_id"] ==
profesional_id:
            return False, "El profesional ya tiene un turno en esa fecha y
horario."
        if turno["hora"] == hora and turno["paciente_id"] == paciente_id:
            return False, "El paciente ya tiene un turno en esa fecha y
horario."
```

Ahora los controles de los datos se delegaron en funciones auxiliares (validar_fecha, validar_horario), y la verificación de superposición de turnos se unificó en una única pasada mediante buscar_conflicto para preservar el orden exacto de los mensajes de error de la versión inicial:

Python
```
conflicto = buscar_conflicto(
    datos["turnos"], paciente_id, profesional_id, fecha, hora
)
if conflicto is not None:
    return False, conflicto
```

En main.py, extrajimos una función por cada opción del menú (opcion_registrar_paciente, opcion_listar_pacientes, opcion_reservar_turno, opcion_cancelar_turno, opcion_consultar_agenda y opcion_consultar_historial) y reemplazamos la cadena de if/elif por el diccionario OPCIONES:

Python
```
accion = OPCIONES.get(opcion)
if accion is None:
    print("Opción inválida.")
else:
    accion(datos)
```

Con este cambio, main() bajó de 91 a 20 líneas y los controles de la reserva quedaron desacoplados para poder probarse por separado, resolviendo simultáneamente H1 y H5 sin alterar las respuestas de la consola.

Verificación: la demo completa, los casos de borde y el caso de doble conflicto dieron la misma salida y el mismo archivo final que antes del cambio, y pasan las 19 pruebas automatizadas en la CI, Black y flake8. Commit 0baf95f, PR #23, revisado e integrado por Agustín.

Refactoring 3: módulo de persistencia (H3)

En la versión inicial, hospital/gestion.py mezclaba las reglas de negocio de los turnos con las funciones cargar_datos y guardar_datos, importando directamente json y shutil. Antes, gestion.py abría y copiaba los archivos junto a las operaciones del dominio:

```python
def cargar_datos(ruta, ruta_ejemplo):
    if not ruta.exists():
        shutil.copyfile(ruta_ejemplo, ruta)
    with open(ruta, encoding="utf-8") as archivo:
        return json.load(archivo)


def guardar_datos(datos, ruta):
    with open(ruta, "w", encoding="utf-8") as archivo:
        json.dump(datos, archivo, ensure_ascii=False, indent=2)
```

Mediante el refactoring Move Method trasladamos las funciones de lectura y escritura al módulo `persistencia.py`. De este modo, `main.py` efectúa las llamadas directamente sobre este componente separado, logrando que `gestion.py` se concentre de forma exclusiva en las reglas del negocio sin interactuar con el archivo de datos.

Verificación: la demo completa y los dieciséis casos de borde dieron la misma salida y el mismo archivo final que antes del cambio. Además probamos aparte un archivo dañado: el aviso es el mismo y el archivo no se sobrescribe. Pasan Black y flake8. Commit 21a5e2c, PR #21, revisado e integrado por Bruno. Las pruebas automatizadas todavía no existían; las 20 pasan sobre la versión final.

Refactoring 4: objeto de solicitud de turno (H6)

En la versión previa, reservar_turno recibía cinco parámetros (datos, paciente_id, profesional_id, fecha, hora) y buscar_conflicto recibía la lista de turnos junto con los mismos cuatro datos sueltos. Esos cuatro valores representan una misma intención de

reserva que viaja junta desde la consola hasta las validaciones del dominio. Antes, las firmas y llamadas pasaban los cuatro parámetros por separado:

```python
def buscar_conflicto(turnos, paciente_id, profesional_id, fecha, hora):
    ...

def reservar_turno(datos, paciente_id, profesional_id, fecha, hora):
    ...
    conflicto = buscar_conflicto(
        datos["turnos"], paciente_id, profesional_id, fecha, hora
    )
```

Aplicamos Introduce Parameter Object definiendo en hospital/gestion.py la clase inmutable SolicitudTurno con @dataclass(frozen=True), y adaptamos reservar_turno, buscar_conflicto y opcion_reservar_turno en main.py para construir y pasar una única instancia:

```python
@dataclass(frozen=True)
class SolicitudTurno:
    paciente_id: int
    profesional_id: int
    fecha: str
    hora: str


def reservar_turno(datos, solicitud: SolicitudTurno):
    ...
    conflicto = buscar_conflicto(datos["turnos"], solicitud)
```

El turno guardado en datos["turnos"] se sigue almacenando como diccionario para mantener intacto el formato del archivo JSON y la compatibilidad con la agenda, el historial y la persistencia.

Verificación: adaptamos cómo las pruebas llaman a reservar_turno, sin cambiar ningún resultado esperado, y pasan las 20 en la CI, Black y flake8. La demo completa, los dieciséis casos de borde y el caso de doble conflicto dieron la misma salida y el mismo

archivo final que antes del cambio. Commit 8879924, PR #26, revisado e integrado por Bruno.

Mejora adicional en la revisión final: escritura atómica con archivo temporal (H9)

En el PR #25 reforzamos guardar_datos en hospital/persistencia.py para que escriba primero en un archivo temporal (local.json.tmp) y reemplace al archivo original mediante os.replace. Si ocurre cualquier error durante la serialización, el bloque except BaseException elimina el archivo temporal y conserva intacto el archivo anterior.

No lo contamos como refactoring porque cambia lo que pasa cuando el guardado falla: antes el archivo quedaba cortado y ahora queda como estaba. Por eso tuvo su propia issue y su propia prueba, que falla sobre la versión anterior y pasa con el cambio. Pasan las 20 pruebas en la CI, Black y flake8, y la salida del programa es la misma. Commit 1bbe682, issue #24, PR #25, revisado e integrado por Santiago.

6. Pruebas o verificaciones realizadas

> Etapa 6. Pruebas y verificación del comportamiento

> *Definir casos de prueba o evidencias de ejecución antes/después. Explicar cómo se verificó que el refactoring no alteró negativamente el comportamiento externo.*

Para comprobar que los refactorings no cambian el comportamiento usamos pruebas de caracterización: casos que registran lo que la versión inicial hace hoy y que tienen que seguir dando lo mismo después de cada cambio. Se escriben con unittest, con datos fijos y archivos temporales, para poder repetirlas sin tocar los datos de la demo. Cubren estos grupos de casos:

Grupo	Casos
Pacientes	Alta con los espacios recortados, nombres iguales con identificadores distintos, nombre vacío o con solo espacios y listado ordenado por número.
Reservas	Reserva válida en cada una de las seis franjas y ocho entradas inválidas: paciente o profesional inexistente, tres fechas inválidas y tres horarios fuera de las franjas.
Conflictos	Profesional ocupado, paciente ocupado, reservas permitidas en otra franja, otra fecha o con otro paciente y profesional, y el mensaje cuando hay dos conflictos.
Cancelaciones	Cancelación válida, franja liberada, identificador inexistente y turno ya cancelado.
Consultas	Agenda vacía o con reservas, historial con activos y cancelados, y orden de los resultados.
Persistencia	Guardar y recargar sin perder datos, archivo ausente (se crea desde el ejemplo), JSON dañado sin sobrescribir el archivo y guardado fallido sin tocar el archivo anterior (H9).
Operaciones rechazadas	Comprobar que no cambian los datos en memoria y, en la persistencia, que el archivo queda igual.

También hay una prueba de integración que recorre reservar, guardar, recargar y consultar. Si un refactoring cambia una firma interna, adaptamos cómo la prueba llama al código, pero no el resultado esperado. Las pruebas, junto con black --check . y flake8 .,

corren automáticamente en GitHub Actions (.github/workflows/ci.yml, integrado en el PR #22) en cada push y en cada Pull Request, actuando como control obligatorio antes de integrar cambios en main.

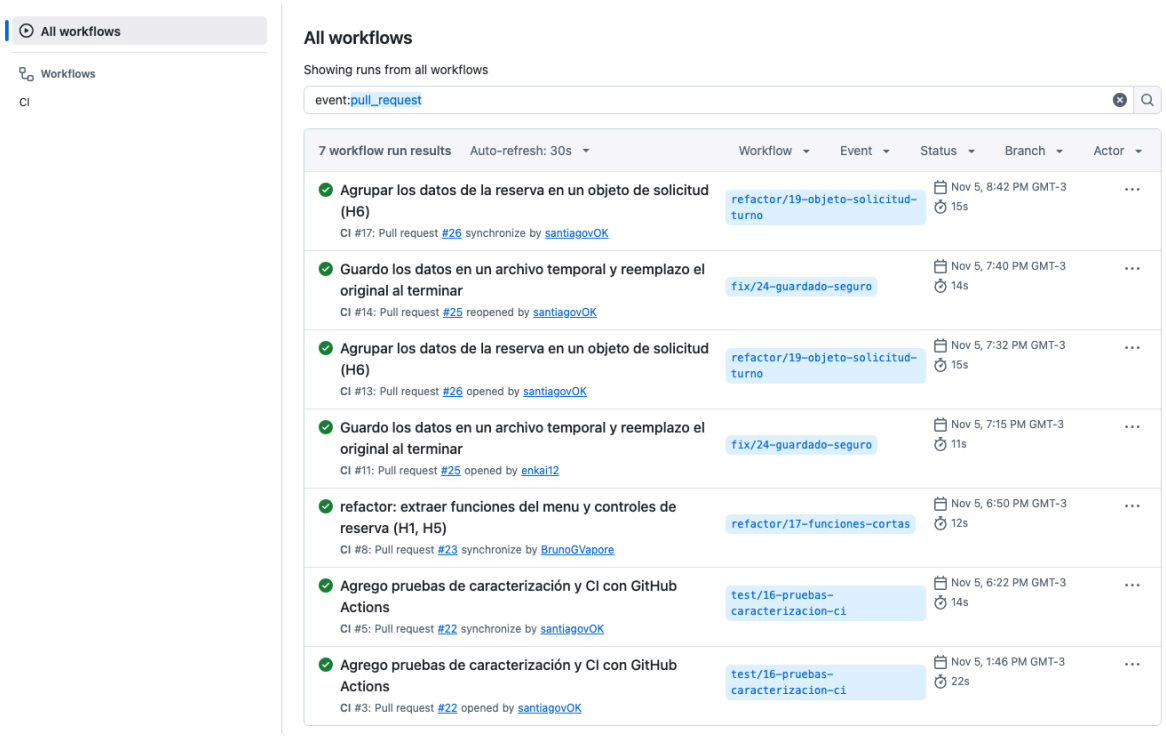
Además, en cada refactoring comparamos la salida completa del programa antes y después del cambio. Corremos la demo completa y dieciséis casos de borde sobre las dos versiones y comparamos lo que muestra la consola y el archivo que queda guardado. En H2, las 349 líneas de salida y el archivo final fueron idénticos.

La suite final cuenta con 20 pruebas automatizadas (12 en test_turnos.py, 5 en test_persistencia.py y 3 en test_pacientes.py), con subtests para las 6 franjas horarias y los 8 casos de entradas inválidas, y una prueba del mensaje cuando hay dos conflictos. Se ejecuta localmente con el comando:

```
python -m unittest discover -s tests -v
```

Las pruebas se escribieron sobre la versión inicial, en paralelo con H2 y H3, y las 18 primeras pasaron sobre ese código sin cambios. Por eso H2 y H3 se comprobaron solo comparando la salida. Desde el PR #22, la integración continua corrió las pruebas en cada Pull Request: 19 en el de H1 (#23) y 20 en los de H9 (#25) y H6 (#26), todas en verde, igual que Black y flake8.

En la versión final (commit 465cc5e) pasan las 20 pruebas, y la demo completa, los dieciséis casos de borde y el caso de doble conflicto dan la misma salida que la versión inicial: 403 líneas iguales y el mismo archivo guardado. También repetimos el análisis de SonarQube con la misma configuración. Pasó de cuatro problemas de mantenibilidad y 99 minutos de deuda estimada a ninguno, y la complejidad cognitiva total bajó de 144 a 76; ninguna función supera el máximo de 15. Igual que en el diagnóstico, la herramienta no ve los problemas de diseño entre módulos, así que H4, H7, H8, H10 y H11 siguen como mejoras futuras. La salida completa de las pruebas y la captura de la integración continua están en docs/pruebas.md del repositorio.



Las ejecuciones de la integración continua en los Pull Request de las pruebas (#22), de H1 (#23), de H9 (#25) y de H6 (#26), todas en verde.

7. Code review y checklist técnico

> Etapa 7. Code review profesional

Elaborar un checklist técnico y aplicarlo sobre el repositorio. Registrar observaciones, feedback profesional, correcciones realizadas y criterios de aceptación técnica.

Para revisar cada Pull Request usamos este checklist, con preguntas concretas para nuestro sistema.

Categoría	Qué revisamos
Lógica y funcionalidad	Si respeta las reglas de reserva y cancelación, si cubre los casos de borde (fecha inválida, franja ocupada, turno ya cancelado) y si conserva el comportamiento de la versión inicial.
Legibilidad	Si los nombres dicen lo que hacen y si cada función se entiende sin tener que preguntarle a quien la escribió.
Diseño	Si cada función o clase tiene una sola responsabilidad, si las reglas de turnos quedan separadas de la consola y del JSON y si hay lógica repetida.
Manejo de errores	Si los errores se informan con un mensaje claro y si hay excepciones capturadas que se pierden sin aviso.
Seguridad y datos	Si se validan las entradas, si se usan solo datos ficticios y si el archivo de trabajo local queda fuera de Git.
Rendimiento	Si se lee o se escribe el archivo más veces de las necesarias.
Pruebas	Si hay pruebas para lo que cambió y si pasan todas.

Un Pull Request se acepta cuando cumple estos criterios de aceptación técnica:

- Pasan las pruebas, Black y flake8.
- No rompe funcionalidades existentes ni cambia las reglas acordadas, salvo que el Pull Request lo diga.
- Respeta los principios de diseño y las convenciones del proyecto.
- Maneja los errores sin ocultarlos.
- No agrega deuda técnica innecesaria.

- Lo revisó otro integrante y sus observaciones quedaron resueltas o justificadas.

Cada observación la escribimos en tres partes: qué vemos en el código, por qué importa y qué proponemos.

El checklist y los criterios también están en docs/checklist-code-review.md del repositorio. Estas son las observaciones que registramos en los Pull Request:

Pull Request	Revisó	Qué observó	Resultado
#7. Entorno de desarrollo	Santiago	Probó la instalación en su equipo, comprobó que Black y flake8 no se contradicen y que el README aclara que el programa no necesita dependencias para ejecutarse.	Aprobado sin cambios.
#8. Base funcional	Bruno	Recorrió los siete pasos de la demo y adjuntó una captura de cada uno. Comprobó las reglas, la persistencia y las validaciones.	Aprobado sin cambios. Sus capturas son la evidencia de la sección 1.
#12. Diagnóstico en el repositorio	Agustín	Comparó cada dato con el código y encontró detalles: el nombre de un registro de decisión, etiquetas de code smells distintas de las del informe, la explicación de la nota A de SonarQube y el error que produce un archivo con [].	Aprobado con comentarios para no frenar el trabajo. Las correcciones se revisaron e integraron en el PR #14.
#14. Ajustes del diagnóstico	Bruno	Verificó que las etiquetas de H6 y H8, la explicación del cálculo de SonarQube y el error TypeError de H4 coincidieran exactamente entre docs/diagnostico.md y el informe.	Aprobado e integrado.
#20. Búsqueda única (H2)	Bruno	Comprobó que buscar_por_id reemplace los siete bucles en gestion.py, corte al encontrar el elemento, mejore los nombres de variables y mantenga intacto el caso de borde de H11.	Aprobado e integrado.

Pull Request	Revisó	Qué observó	Resultado
#21. Módulo de persistencia (H3)	Bruno	Verificó que gestion.py ya no importe json ni shutil, que persistencia.py conserve el manejo de local.json y ejemplo.json, y que pasen Black, flake8 y la comparación de la salida.	Aprobado e integrado.
#22. Pruebas de caracterización y CI	Agustín	Verificó que las pruebas cubran pacientes, reservas, conflictos, cancelaciones, agenda, historial y persistencia usando tempfile, y que el workflow de GitHub Actions ejecute unittest, black y flake8. Encontró que la prueba de persistencia importaba cargar_datos y guardar_datos desde gestion.py, aunque desde H3 están en persistencia.py; la CI había salido verde porque corrió contra el main anterior a H2 y H3.	Santiago actualizó la rama con main, corrigió el import y sumó una prueba del mensaje cuando hay dos conflictos. Aprobado e integrado.
#23. Funciones cortas y fecha única (H1 y H5)	Agustín	Detectó con los casos de borde que separar profesional_ocupado y paciente_ocupado invertía el mensaje de error cuando coincidían ambos conflictos en distintos turnos, y propuso unificar el recorrido en buscar_conflicto.	Corregido en la rama, aprobado e integrado.
#25. Guardado con archivo temporal (H9)	Santiago	Comprobó el reemplazo atómico con os.replace, la limpieza del archivo .tmp ante excepciones y la nueva prueba unitaria que fuerza un TypeError sin dañar local.json.	Aprobado e integrado.
#26. Objeto SolicitudTurno (H6)	Bruno	Verificó que SolicitudTurno sea inmutable (frozen=True), que reservar_turno y buscar_conflicto reciban el objeto sin cambiar el diccionario persistido en JSON y que pasen las 20 pruebas.	Aprobado e integrado.
#33. Documentación de los refactorings	Agustín	Comparó cada fragmento de código con el commit que cita y los SHA con los reales. Encontró títulos de issues que no coincidían con GitHub, etiquetas que el diagnóstico ya no	Aprobado con comentarios para no frenar el trabajo. Las correcciones se

Pull Request	Revisó	Qué observó	Resultado
		usa y el nombre del refactoring de H3 distinto del informe.	revisaron e integraron en el PR #34.
#34. SonarQube final y diagnóstico	Santiago	Comprobó el cuadro comparativo contra las capturas, los hallazgos que quedan como mejoras futuras y los enlaces internos. Pidió una captura para cada cifra de la tabla.	Se sumaron las capturas de complejidad, líneas y funciones. Aprobado e integrado.
#35. README final	Bruno	Comprobó que el README describe el estado final de main, con los módulos reales, las tecnologías, los comandos para Linux, macOS y Windows y la opción 0, y que los enlaces a docs/ y la tabla de tags coinciden con el informe.	Aprobado e integrado.
#36. Pruebas y revisión final	Santiago	Pidió sumar al registro de pruebas la salida de Black y flake8, y al checklist la tabla de revisiones de los Pull Request con la revisión del #26.	Bruno sumó las dos cosas y corrigió la cronología de las pruebas. Aprobado e integrado.

Por ejemplo, una de las observaciones del PR #12, ordenada en las tres partes del checklist:

- Qué vemos: en H6 y H8, el repositorio usa las etiquetas Data Clumps y Feature Envy, que el informe no usa.
- Por qué importa: el repositorio y el informe tienen que decir lo mismo sobre cada hallazgo.
- Qué proponemos: usar las mismas etiquetas que el informe, Long Parameter List y acoplamiento.

Al terminar, aplicamos el checklist sobre el código final (commit 465cc5e). El detalle, con las cuatro observaciones, está en docs/revision-final.md del repositorio.

Categoría	Resultado sobre la versión final
Lógica y funcionalidad	Cumple: la salida es igual a la de la versión inicial (403 líneas y el mismo archivo final).
Legibilidad	Cumple: ninguna función pasa de 35 líneas y main() tiene 20. Queda un lambda t en historial_paciente.
Diseño	Cumple: las reglas no tocan el archivo y la búsqueda está en una sola función. La consola todavía lee el diccionario interno (H8).
Manejo de errores	Cumple en el uso normal: ningún error se pierde sin aviso. Siguen los fallos de H4 con archivos de otra forma.
Seguridad y datos	Cumple: se validan las entradas, los datos son ficticios y local.json queda fuera de Git.
Rendimiento	Cumple: el archivo se lee una vez y se escribe solo cuando una operación se acepta.
Pruebas	Cumple: pasan las 20 pruebas, Black y flake8. La consola se comprueba con la comparación de la salida, no con pruebas automatizadas.

Ninguna observación frena la entrega: H4 y H8 ya figuran como mejoras futuras, y el nombre del lambda queda para un próximo cambio.

Otro ejemplo de observación técnica, registrada en la revisión del PR #23 (H1):

- Qué vemos: antes la reserva recorría los turnos una sola vez y devolvía el conflicto del primer turno que coincidía; al separarlo en profesional_ocupado y paciente_ocupado, revisa todos los turnos del profesional primero y los del paciente después.
- Por qué importa: si el paciente está ocupado en un turno y el profesional en otro a la misma hora, cambia el texto del mensaje de rechazo respecto de la versión inicial.
- Qué proponemos: usar una única función buscar_conflicto(turnos, paciente_id, profesional_id, fecha, hora) que recorra la lista en una sola pasada, preservando el comportamiento exacto.

8. Conclusiones y mejoras futuras

> Etapa 8. Conclusiones y aprendizaje

• *Conclusiones grupales e individuales breves.*

Conclusiones grupales

Trabajar sobre un sistema que ya funcionaba desde la mirada del usuario nos permitió distinguir en la práctica entre calidad externa y calidad interna. Aprendimos que refactorizar de forma segura exige poder comparar el comportamiento antes y después de cada cambio, con pruebas de caracterización y casos de borde que detecten cualquier diferencia sutil en las salidas, como nos ocurrió al extraer la validación de conflictos en el PR #23. También comprobamos que herramientas como SonarQube son muy útiles para medir la complejidad cognitiva y detectar literales repetidos, pero no reemplazan la lectura crítica del código ni las revisiones cruzadas para identificar problemas de diseño como la mezcla de responsabilidades (H3) o el acoplamiento entre capas (H8). Como límite del trabajo, mantuvimos la estructura de diccionarios en memoria y el retorno de tuplas (ok, resultado) para acotar el alcance de los cambios y no introducir regresiones.

Entre las decisiones, sostuvimos que un refactoring no cambia el comportamiento: por eso conservamos el caso de H11 y tratamos H9 como una corrección aparte, con su propia issue y su propia prueba. También aplicamos solo los principios que tenían sentido en un programa chico y explicamos por qué LSP, ISP y DIP no correspondían. La dificultad principal fue coordinar ramas en paralelo: las pruebas se escribieron al mismo tiempo que los primeros refactorings, así que H2 y H3 se comprobaron solo comparando la salida, y en el PR #22 la CI había pasado contra una versión vieja de main. Desde entonces exigimos que cada rama esté al día con main antes de integrarla.

Mejoras futuras

Estos hallazgos del diagnóstico no entraron en los refactorings y quedan como próximos pasos:

- Validar la estructura del archivo al cargar y atrapar los errores previstos con un aviso claro (H4).
- Informar los errores con excepciones propias en lugar de devolver (False, mensaje) (H7).
- Que la consola use solo funciones de consulta y no lea el diccionario interno (H8).
- Representar el turno y su estado con tipos propios (H10).

- Que la agenda marque la franja como ocupada aunque falte el paciente y que se validen las referencias al cargar (H11).

Conclusiones individuales

Agustín: Armar la base funcional, el diagnóstico y los refactorings de búsqueda única (H2), separación de persistencia (H3) y guardado atómico (H9) me permitió ver cómo pequeños cambios bien acotados mejoran la cohesión del código. La revisión cruzada de Pull Requests nos ayudó a detectar diferencias sutiles entre la documentación y el comportamiento real antes de llegar a la rama principal.

Bruno: Encargarme de la documentación de los hallazgos y los ADRs en el repositorio, junto con el refactoring de extracción de funciones en el menú y las validaciones de reserva (H1 y H5), me demostró la importancia de bajar la complejidad cognitiva sin alterar el comportamiento observable. El feedback recibido en el code review del PR #23 fue clave para comprender que un refactoring debe preservar hasta el orden exacto de los mensajes en los casos de borde.

Santiago: Diseñar la suite de pruebas de caracterización con unittest, configurar el flujo de integración continua en GitHub Actions y aplicar la técnica Introduce Parameter Object con SolicitudTurno (H6) me permitió comprobar el valor de automatizar los controles. Tener 20 pruebas corriendo en cada Pull Request nos dio la tranquilidad necesaria para modificar las firmas internas sabiendo que las reglas del negocio seguían intactas.

9. Anexos y enlaces

9.1 Repositorio GitHub

• Enlace al repositorio GitHub asociado, el cual debe cumplir con la siguiente estructura:

• Proyecto completo con los tags obligatorios de estado general: estado-inicial (para la entrega del sistema base) y estadorefactorizado (para el estado final del código).

• Tags específicos en los commits clave para identificar el hito de cada etapa: etapa-diagnostico, etapa-refactoring, etapa-pruebas y etapa-docs.

• README.md con descripción, instalación/uso, funcionalidades y tecnologías.

• Carpeta /docs o equivalente con checklist de code review, evidencias y decisiones técnicas.

• Historial de commits claro y ordenado.

Repositorio: https://github.com/enkai12/hospital-central-msii

Trabajamos con ramas cortas, una por issue, y cada cambio entra a main por un Pull Request que aprueba otro integrante. main está protegida para que nadie pueda saltear ese paso ni integrar un cambio sin que pase la integración continua. El trabajo está organizado en issues, con un milestone por cada hito.

Tag	Qué marca	Commit
estado-inicial	La versión inicial que analizamos, antes de refactorizar.	e5d6927
etapa-diagnostico	El diagnóstico completo y documentado en el repositorio.	e60e89c
etapa-refactoring	Los refactorings aplicados y comprobados.	0ed7e4b
etapa-pruebas	La verificación final, con las pruebas después de las correcciones.	d712614

Tag	Qué marca	Commit
estadorefactorizado	El código final revisado y comprobado.	d712614
etapa-docs	La documentación final y los enlaces actualizados.	Commit de cierre

El repositorio tiene:

- README.md: descripción, tecnologías, estructura, instalación y uso en cada sistema, reglas, demo, pruebas, documentación y tags.
- main.py y hospital/: la consola y las reglas de la agenda.
- docs/diagnostico.md: la calidad externa e interna, los once hallazgos, el análisis de SonarQube de la versión inicial y de la final, y el estado de cada hallazgo.
- docs/adr/: tres registros de decisiones técnicas (alcance y consola, persistencia en JSON, y flujo de trabajo en Git).
- docs/checklist-code-review.md: el checklist y los criterios de aceptación.
- docs/refactorings.md: una ficha por refactoring, con el código antes y después, los commits y cómo se comprobó, y la corrección de H9 aparte.
- docs/pruebas.md: qué cubre cada archivo de pruebas, el entorno, la salida de las 20 pruebas, de Black y de flake8, y la captura de la integración continua.
- docs/revision-final.md: el checklist aplicado sobre la versión final, con las cuatro observaciones y los criterios de aceptación.
- docs/evidencias/: las capturas de SonarQube de la versión inicial y de la final (pantalla principal, issues, complejidad, líneas y funciones) y la captura de la integración continua.
- tests/: la suite de 20 pruebas de caracterización (pacientes, turnos y persistencia).
- .github/workflows/ci.yml: la integración continua en GitHub Actions, que corre Black, flake8 y las pruebas en cada push y en cada Pull Request.

9.2 Video y participación de los integrantes

• Enlace al video explicativo e identificación clara de la participación y contribución de cada integrante.

Video: https://www.youtube.com/watch?v=JWyaUtaGNEY

Participación de los integrantes:

Agustín: Configuración del entorno inicial y base funcional (PR #7 y #8), ajustes de diagnóstico y evidencias (PR #14), refactorings de búsqueda única H2 (PR #20), módulo de persistencia H3 (PR #21) y guardado atómico H9 (PR #25), el análisis final de SonarQube y el diagnóstico alineado con el informe (PR #34), y la revisión de PRs (#12, #22, #23 y #33).

Bruno: Ejecución y capturas de la demo inicial (PR #8), documentación del diagnóstico H1–H11 y ADRs en el repositorio (PR #12), refactoring de extracción de funciones del menú y controles de reserva H1/H5 (PR #23), la documentación de las pruebas y de la revisión final en el repositorio (PR #36), redacción del informe y revisión técnica de PRs (#14, #20, #21, #26 y #35).

Santiago: Implementación de la suite de pruebas de caracterización y configuración de CI con GitHub Actions (PR #22), refactoring de objeto de solicitud de turno H6 (PR #26), la documentación de los refactorings en el repositorio (PR #33), el README final (PR #35), la creación de las issues del diagnóstico (#9 a #11) y de las del cierre (#27 a #32), y la revisión técnica de PRs (#7, #25, #34 y #36).

9.3 Uso de inteligencia artificial

> • El uso de IA está permitido como apoyo para comprender conceptos, revisar alternativas o mejorar redacción.
>
> • Debe declararse en el informe indicando herramienta utilizada, finalidad y grado de intervención.
>
> • Debe adjuntarse evidencia del intercambio relevante en PDF o anexo.
>
> • La IA no reemplaza la autoría ni la defensa. El equipo debe comprender y justificar todo lo entregado.

Durante el desarrollo del trabajo utilizamos modelos de lenguaje de asistencia conversacional (Gemini y Claude) como apoyo técnico y de redacción.

Se realizaron consultas sobre convenciones de configuración de herramientas de calidad (black, flake8, unittest y GitHub Actions), revisión de alternativas para aplicar los patrones de refactoring (Extract Method, Move Method e Introduce Parameter Object), resolución de dudas sobre el flujo de ramas en Git y revisión de estilo y consistencia del informe técnico.

El grado de intervención fue de asistencia y consulta. Todas las decisiones de diseño, la identificación y priorización de los once hallazgos, la escritura final del código, la validación

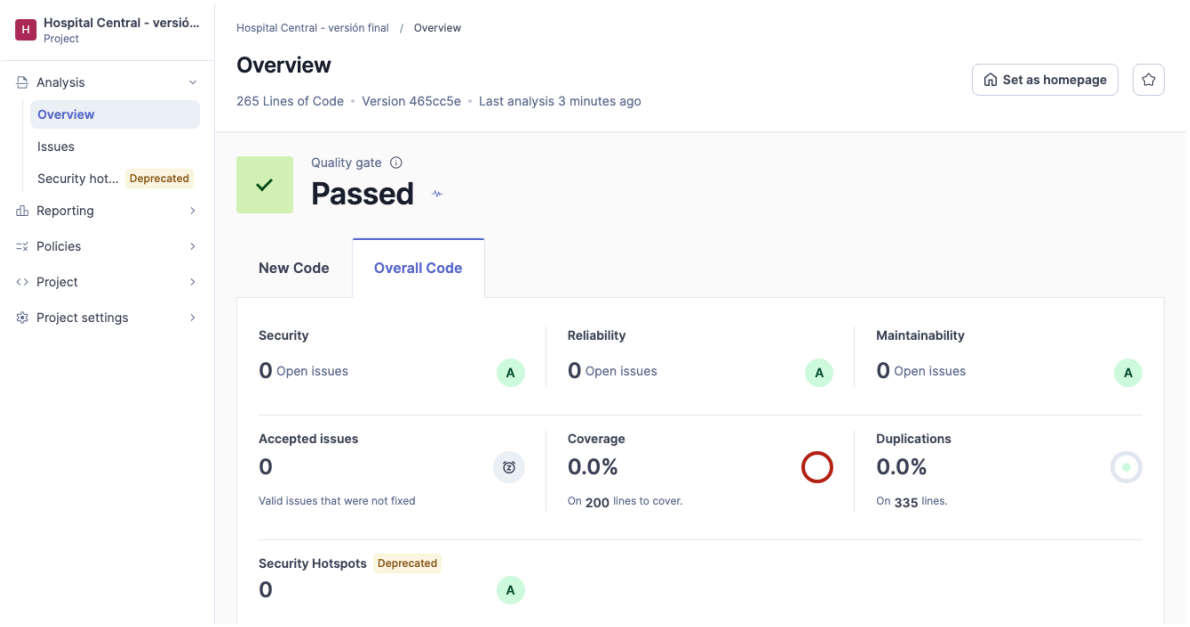
con casos de borde y las revisiones cruzadas en los Pull Requests fueron realizadas, probadas y verificadas íntegramente por los miembros del equipo.

9.4 Evidencias complementarias

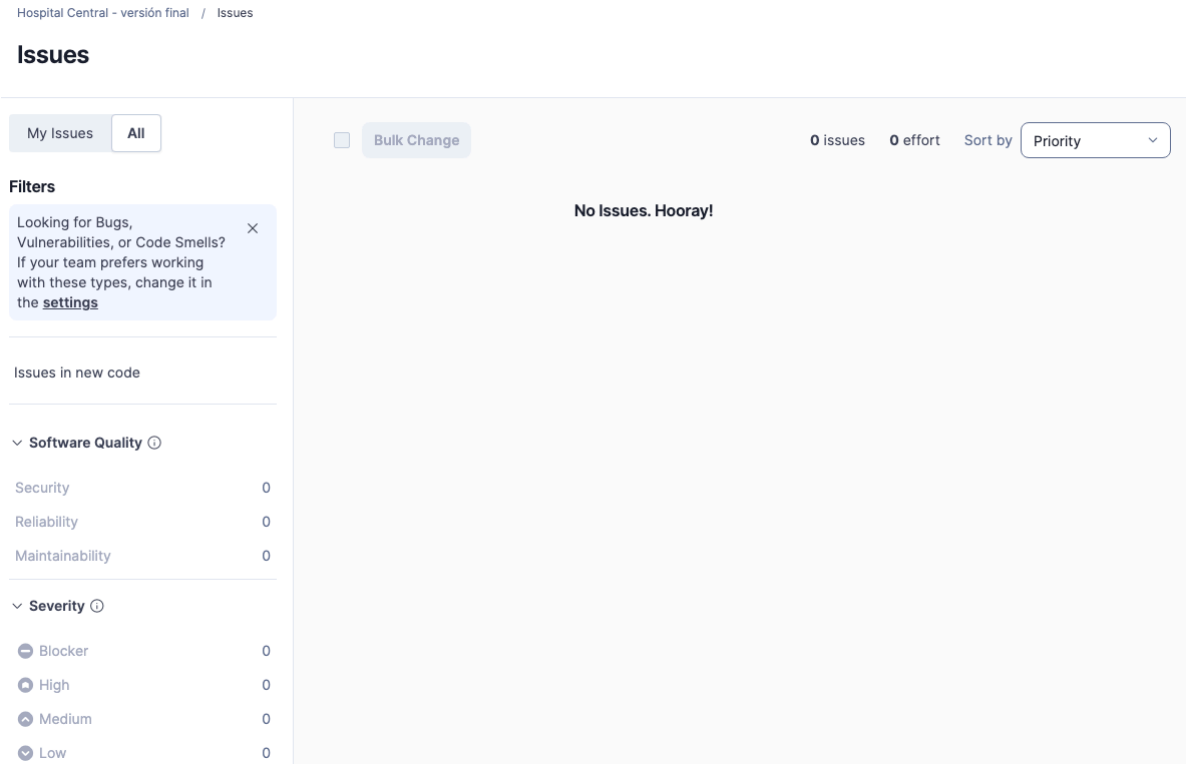
> • Desarrollo de cada etapa obligatoria, respaldado con capturas, tablas, fragmentos de código para el diagnóstico y las mejoras, junto con el registro ordenado y detallado de los prompts utilizados durante las actividades (incluyendo interacciones con herramientas de IA) y sus evidencias complementarias (diagramas, informes de herramientas analíticas, archivos de prueba y documentación adicional).

En el ZIP de evidencias incluimos:

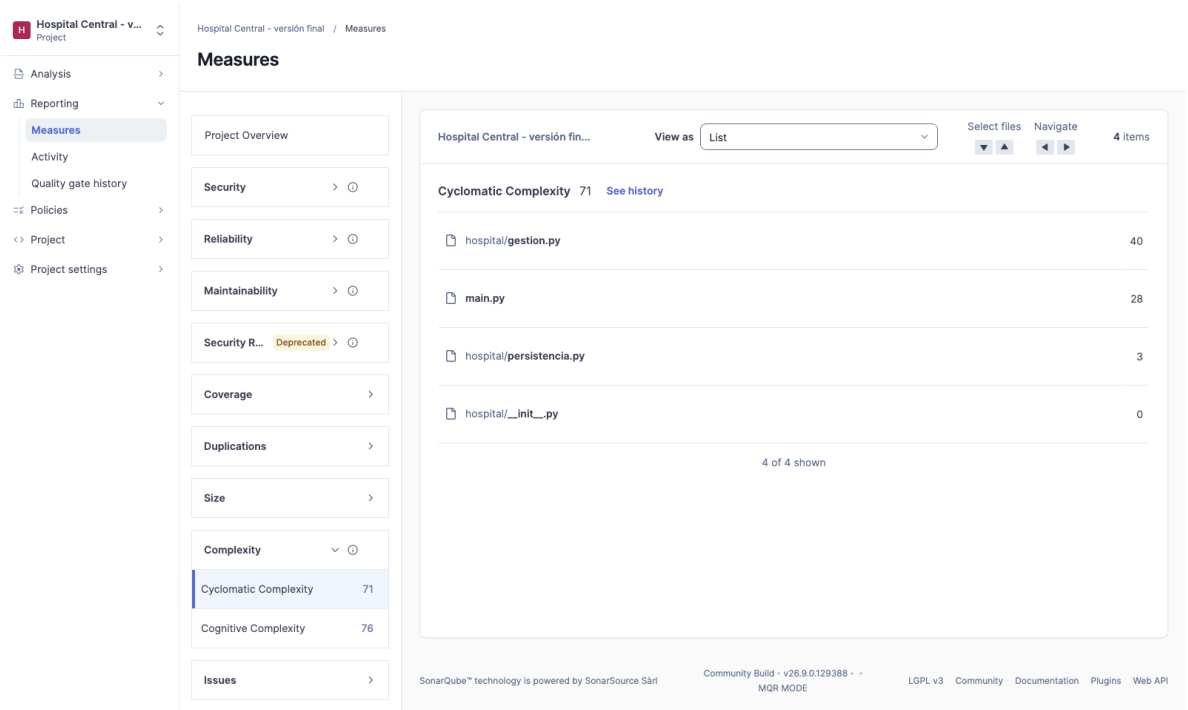
- Las capturas de la demo de la versión inicial (sección 1) y las de SonarQube (sección 3).
- La comparación de la salida antes y después de cada refactoring.
- La salida completa de las 20 pruebas automatizadas, la captura de ejecución del pipeline de GitHub Actions y el informe final de SonarQube.

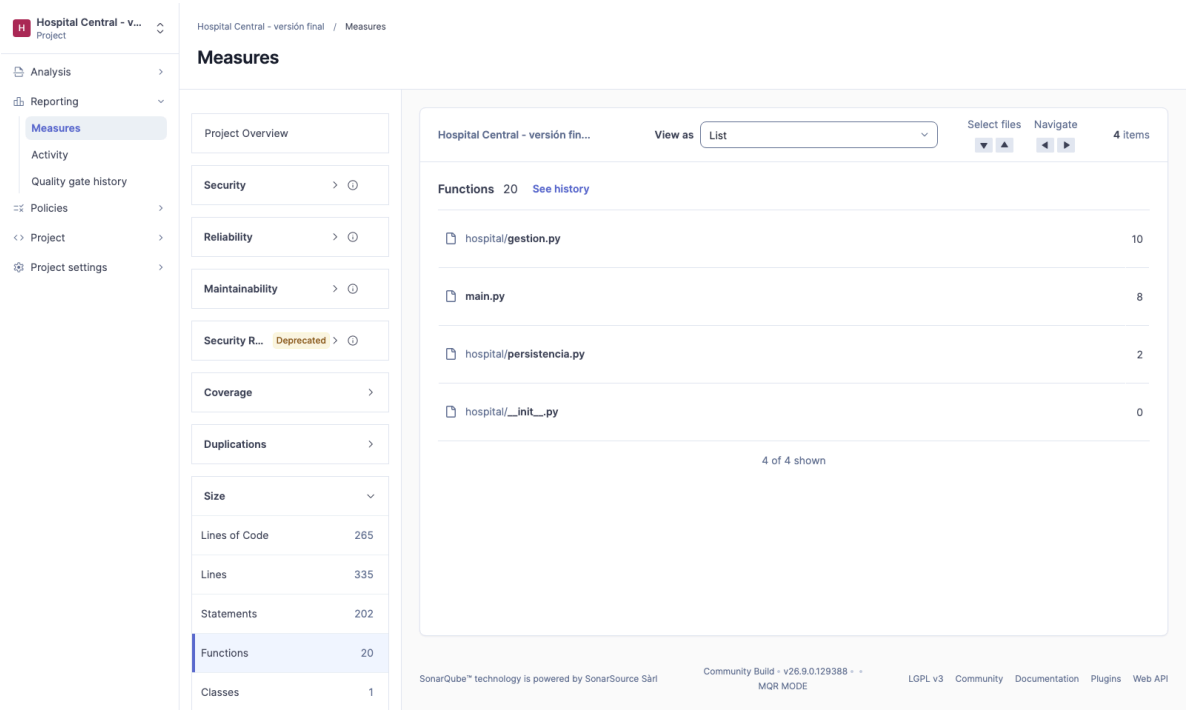


La captura de arriba muestra el resumen del análisis estático de SonarQube sobre el código refactorizado final (commit 465cc5e), evidenciando el cumplimiento exitoso del Quality Gate con cero problemas abiertos.



En la captura de arriba se muestra el panel de incidencias de la versión final tras aplicar los refactorings estructurales, sin problemas abiertos. Los hallazgos de diseño que la herramienta no detecta (H4, H7, H8, H10 y H11) siguen como mejoras futuras.





Las dos capturas de arriba respaldan las métricas actualizadas de líneas de código (265) y distribución de funciones (20) en el estado final del repositorio.

A continuación se muestra la salida por consola de las 20 pruebas automatizadas:

```
test_listar_pacientes_orden_numerico_y_no_muta_original (test_pacientes.TestPacientes.test_listar_pacientes_orden_numerico_y_no_muta_original)
Devuelve pacientes ordenados por ID sin alterar el orden original. ... ok
test_registrar_paciente_con_espacios_y_nombres_duplicados (test_pacientes.TestPacientes.test_registrar_paciente_con_espacios_y_nombres_duplicados)
Registra pacientes recortando espacios y permite nombres duplicados. ... ok
test_registrar_paciente_rechaza_nombres_vacios_o_espacios (test_pacientes.TestPacientes.test_registrar_paciente_rechaza_nombres_vacios_o_espacios)
Rechaza altas con cadenas vacías o solo espacios sin alterar datos. ... ok
test_cargar_crea_destino_desde_ejemplo (test_persistencia.TestPersistencia.test_cargar_crea_destino_desde_ejemplo)
Cargar crea el archivo destino a partir del ejemplo si no existe. ... ok
test_cargar_json_invalido_no_reemplaza_destino (test_persistencia.TestPersistencia.test_cargar_json_invalido_no_reemplaza_destino)
Cargar un archivo con JSON inválido propaga error y preserva archivos. ... ok
test_guardar_con_error_no_modifica_destino (test_persistencia.TestPersistencia.test_guardar_con_error_no_modifica_destino)
Si el guardado falla, el archivo anterior queda igual y sin temporales. ... ok
test_guardar_y_recargar_conserva_datos (test_persistencia.TestPersistencia.test_guardar_y_recargar_conserva_datos)
Guardar y recargar conserva la totalidad de los datos y caracteres. ... ok
test_reserva_persistida_aparece_en_agenda_e_historial (test_persistencia.TestPersistencia.test_reserva_persistida_aparece_en_agenda_e_historial)
Una reserva guardada y recargada se refleja en agenda e historial. ... ok
test_agenda_profesional_estructura_ocupacion_y_cancelacion (test_turnos.TestTurnos.test_agenda_profesional_estructura_ocupacion_y_cancelacion)
Agenda devuelve las 6 franjas ordenadas, refleja ocupación y cancelados. ... ok
test_agenda_profesional_invalida_no_muta_datos (test_turnos.TestTurnos.test_agenda_profesional_invalida_no_muta_datos)
Consultar agenda con profesional inexistente o fecha inválida no muta. ... ok
test_cancelacion_invalida_no_muta_datos (test_turnos.TestTurnos.test_cancelacion_invalida_no_muta_datos)
Cancelar un turno inexistente o ya cancelado falla sin alterar datos. ... ok
Cancelar actualiza a CANCELADO y permite reservar de nuevo esa franja. ... ok
test_conflictos_y_concurrencia_de_franjas (test_turnos.TestTurnos.test_conflictos_y_concurrencia_de_franjas)
Superposiciones se rechazan sin mutar; combinaciones libres se permiten. ... ok
test_historial_paciente_desempate_por_id (test_turnos.TestTurnos.test_historial_paciente_desempate_por_id)
Ante misma fecha y hora, el historial desempata por ID ascendentemente. ... ok
Historial filtra por paciente, incluye cancelados y ordena ... ok
test_precedencia_mensaje_conflictos_en_turnos_distintos (test_turnos.TestTurnos.test_precedencia_mensaje_conflictos_en_turnos_distintos)
El orden de los turnos determina el mensaje ante dos conflictos. ... ok
test_rechazo_entradas_invalidas_no_muta_datos (test_turnos.TestTurnos.test_rechazo_entradas_invalidas_no_muta_datos)
Casos inválidos devuelven False y preservan los datos intactos. ... ok
test_reserva_valida_crea_turno_y_avanza_contador (test_turnos.TestTurnos.test_reserva_valida_crea_turno_y_avanza_contador)
Una reserva válida crea el registro ACTIVO y avanza proximo_turno. ... ok
test_reserva_valida_en_todas_las_franjas (test_turnos.TestTurnos.test_reserva_valida_en_todas_las_franjas)
Cada una de las franjas horarias válidas permite reservar con éxito. ... ok

Ran 20 tests in 0.020s
OK
```